

paso a paso ejercicio de auditoría API RESTful (Postman + Burp Suite)

Objetivo del ejercicio

Auditar una API RESTful corriendo en <http://localhost:3000> en busca de:

- Fallas de control de acceso
- JWTs mal protegidos
- Manipulación de privilegios desde el cliente

1. Preparación del entorno

A. Clona el proyecto e inicia el entorno

```
git clone https://github.com/blandskron/api-rest-auditoria.git
```

```
cd api-rest-auditoria
```

B. Crea los archivos necesarios:

```
entrypoint.sh
```

```
#!/bin/sh
```

```
echo "Ejecutando migraciones..."
```

```
npx prisma migrate deploy
```

```
echo "Ejecutando seed..."
```

```
npx ts-node prisma/seed.ts || echo "Seed no encontrado. Continuando..."
```

```
echo "Iniciando servidor..."
```

```
node dist/server.js
```

.dockerignore

node_modules

dist

Dockerfile

docker-compose.yml

.dockerignore

build-and-push.sh

C. Dockerfile (final)

FROM node:18-alpine

WORKDIR /app

COPY package*.json ./

COPY tsconfig.json ./

COPY prisma ./prisma

COPY src ./src

COPY entrypoint.sh ./entrypoint.sh

RUN npm install

RUN npm install -D typescript ts-node

RUN npx prisma generate

RUN npx tsc

RUN chmod +x entrypoint.sh



EXPOSE 3000

ENTRYPOINT ["/entrypoint.sh"]

D. docker-compose.yml

services:

api:

image: blandskron/api-auditoria:latest

container_name: auditoria-api

ports:

- "3000:3000"

environment:

- PORT=3000

- JWT_SECRET=supersecreto123

- DATABASE_URL=file:./dev.db

2. Ejecución del entorno

docker-compose up

Accede a la API en: <http://localhost:3000/api>

3. Parte 1: Auditoría funcional con Postman

Paso 1: Login

- **POST** `/api/auth/login`

```
{  
  "email": "user@example.com",  
  "password": "User1234"  
}
```

Paso 2: Guarda el JWT recibido

Guarda el token que recibiste.

Paso 3: Solicita tu perfil

- **GET** `/api/users/me`
- Header:

Authorization: Bearer <TOKEN>

Deberías ver tu perfil: `{"id": 1, "name": "...", "role": "user"}`

4. Parte 2: Interceptar con Burp Suite

Paso 1: Configura Burp

- Proxy: `127.0.0.1:8080`
- Configura Postman o el navegador para redirigir tráfico

Paso 2: Intercepta **GET /api/users**

- Modifica el token en jwt.io:

```
{  
  "id": 1,  
  "name": "User",  
  "role": "admin"  
}
```

- Firma el nuevo JWT (si conoces el secreto)

Paso 3: Reenvía con Burp Suite

Prueba el nuevo token con:

- **GET /api/users**
- **PUT /api/users/2/role** con:

```
{ "role": "admin" }
```

5. Parte 3: Validación de control de acceso

A. Con token original

- **GET /api/users** → ¿403?

B. Con token modificado

- ¿200? → ¡Vulnerabilidad grave!

Entregables esperados

1. Capturas de pantalla de:
 - Token recibido
 - GET `/users/me`
 - GET `/users` fallido y exitoso
 - PUT `/users/:id/role` modificado
2. Descripción paso a paso
3. Vulnerabilidades encontradas
4. 3 medidas de mitigación

Medidas de mitigación

Problema

Solución

Firma JWT débil

Usar secreto fuerte y rotación

JWT confiado sin validar

Verificar firma en cada request

Control de acceso débil

Implementar RBAC real en backend

Preguntas de reflexión

- ¿Qué ocurre si un JWT puede ser modificado sin detección?
- ¿Cuál es la diferencia entre autenticación y autorización?
- ¿Qué controles podrías aplicar a nivel de middleware o roles?